type a *type guard*. A type guard is a block of code in which you narrow the definition of a union type in such a way that TypeScript can infer the narrower type and use that information to type check the code.

For example, one use of a union type is to allow your method to take in multiple different types of arguments, so you might have a method like this, admittedly a little contrived:

```
const logAThing(log: number | string | boolean | symbol) {
}
```

We want to treat the `log` argument differently based on its type, which likely involves calling methods that only make sense for the actual type of the argument, not the union type, so we want to be able to have a type guard based on the type. The way in which we can create a type guard depends on what kind of types we have making up our union types.

In this case, where the types are all JavaScript primitive types, you can use the keyword `typeof` as a type guard.

Using `typeof` only works on the four types shown in this snippet:

```
const logAThing(log: number | string | boolean | symbol) {
  if (typeof log === string) {
    logString(log)

  } else if (typeof log === boolean) {
    logBoolean(log)
  }
  // ...and so on
}
```

Inside the first `if` statement, TypeScript is able to treat `log` as a string, and inside the second, TypeScript will treat `log` as a boolean. You can also do a negative check, which would look like `typeof !== symbol`, if there's some reason why that makes sense to do so.

Using `typeof` has a serious limitation, however. It only acts as a type guard with those four primitive types. Specifically, `typeof` does not allow you to differentiate